

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Автоэнкодеры»

Выполнил:

Студент 4 курса

Группы ИИ-26

Ковальчук А.И.

Проверил:

Никитин С.К.

Брест 2026

Цель: научиться применять автоэнкодеры для осуществления визуализации данных и их анализа

Общее задание:

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент с использованием нейросетевой модели автоэнкодера (с двумя и тремя нейронами в среднем слое).
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров.
3. Реализовать метод t-SNE для визуализации данных (использовать также 2 и 3 компонента), построить соответствующую визуализацию.
4. Применить к данным метод PCA (2 и 3 компонента), реализованный в ЛР №1, сделать выводы.
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Вариант 7

№	Выборка	Класс
7	Mushroom	poisonous

Программная реализация:

```
import pandas as pd
```

```
import numpy as np
```

```
import tensorflow as tf
```

```
import matplotlib.pyplot as plt
```

```
from keras import Model, Sequential
```

```
from keras.layers import Dense, Input
```

```
from keras.optimizers import Adam
```

```

from keras.callbacks import EarlyStopping

url =
"https://archive.ics.uci.edu/ml/machine-learning-databases
/mushroom/agaricus-lepiota.data"

columns = ['class', 'cap-shape', 'cap-surface',
'cap-color', 'bruises', 'odor',
          'gill-attachment', 'gill-spacing', 'gill-size',
'gill-color',
          'stalk-shape', 'stalk-root',
'stalk-surface-above-ring',
          'stalk-surface-below-ring',
'stalk-color-above-ring',
          'stalk-color-below-ring', 'veil-type',
'veil-color', 'ring-number',
          'ring-type', 'spore-print-color', 'population',
'habitat']

df = pd.read_csv(url, names=columns)

y = df['class'].map({'p': 1, 'e': 0}).values

X_df = pd.get_dummies(df.drop('class', axis=1),
dtype=np.float32)

X = X_df.values

input_dim = X.shape[1]

```

```

class AutoEncoder(Model):
    def __init__(self, input_dim, latent_dim,
hidden_layers=[64, 32, 16]):

        super().__init__()

        encoder_seq = [Input(shape=(input_dim,))]

        for units in hidden_layers:

            encoder_seq.append(Dense(units,
activation='relu'))

            encoder_seq.append(Dense(latent_dim,
activation='linear'))

        self.encoder = Sequential(encoder_seq)

        decoder_seq = [Input(shape=(latent_dim,))]

        for units in reversed(hidden_layers):

            decoder_seq.append(Dense(units,
activation='relu'))

            decoder_seq.append(Dense(input_dim,
activation='sigmoid'))

        self.decoder = Sequential(decoder_seq)

    def call(self, x):

        latent_x = self.encoder(x)

        decoded_x = self.decoder(latent_x)

```

```
        return decoded_x

from sklearn.manifold import TSNE

from sklearn.decomposition import PCA

import matplotlib.pyplot as plt

print("Вычисление t-SNE 2D...")

tsne_2d = TSNE(n_components=2, random_state=42, n_jobs=-1)

X_tsne_2d = tsne_2d.fit_transform(X)

print("Вычисление t-SNE 3D...")

tsne_3d = TSNE(n_components=3, random_state=42, n_jobs=-1)

X_tsne_3d = tsne_3d.fit_transform(X)

print("Вычисление PCA 2D...")

pca_2d = PCA(n_components=2, random_state=42)

X_pca_2d = pca_2d.fit_transform(X)

print("Вычисление PCA 3D...")

pca_3d = PCA(n_components=3, random_state=42)

X_pca_3d = pca_3d.fit_transform(X)
```

```
fig = plt.figure(figsize=(20, 16))

# 1. График t-SNE 2D

ax1 = fig.add_subplot(221)

scatter1 = ax1.scatter(X_tsne_2d[:, 0], X_tsne_2d[:, 1],
c=y, cmap='coolwarm', alpha=0.6, s=15)

ax1.set_title('t-SNE проекция (2D)')

ax1.legend(*scatter1.legend_elements(), title="Classes")

ax1.grid(True)

# 2. График t-SNE 3D

ax2 = fig.add_subplot(222, projection='3d')

ax2.scatter(X_tsne_3d[:, 0], X_tsne_3d[:, 1], X_tsne_3d[:,
2], c=y, cmap='coolwarm', alpha=0.6, s=15)

ax2.set_title('t-SNE проекция (3D)')

# 3. График PCA 2D

ax3 = fig.add_subplot(223)

scatter3 = ax3.scatter(X_pca_2d[:, 0], X_pca_2d[:, 1],
c=y, cmap='coolwarm', alpha=0.6, s=15)

ax3.set_title('PCA проекция (2D)')

ax3.legend(*scatter3.legend_elements(), title="Classes")

ax3.grid(True)
```

```
# 4. График PCA 3D

ax4 = fig.add_subplot(224, projection='3d')

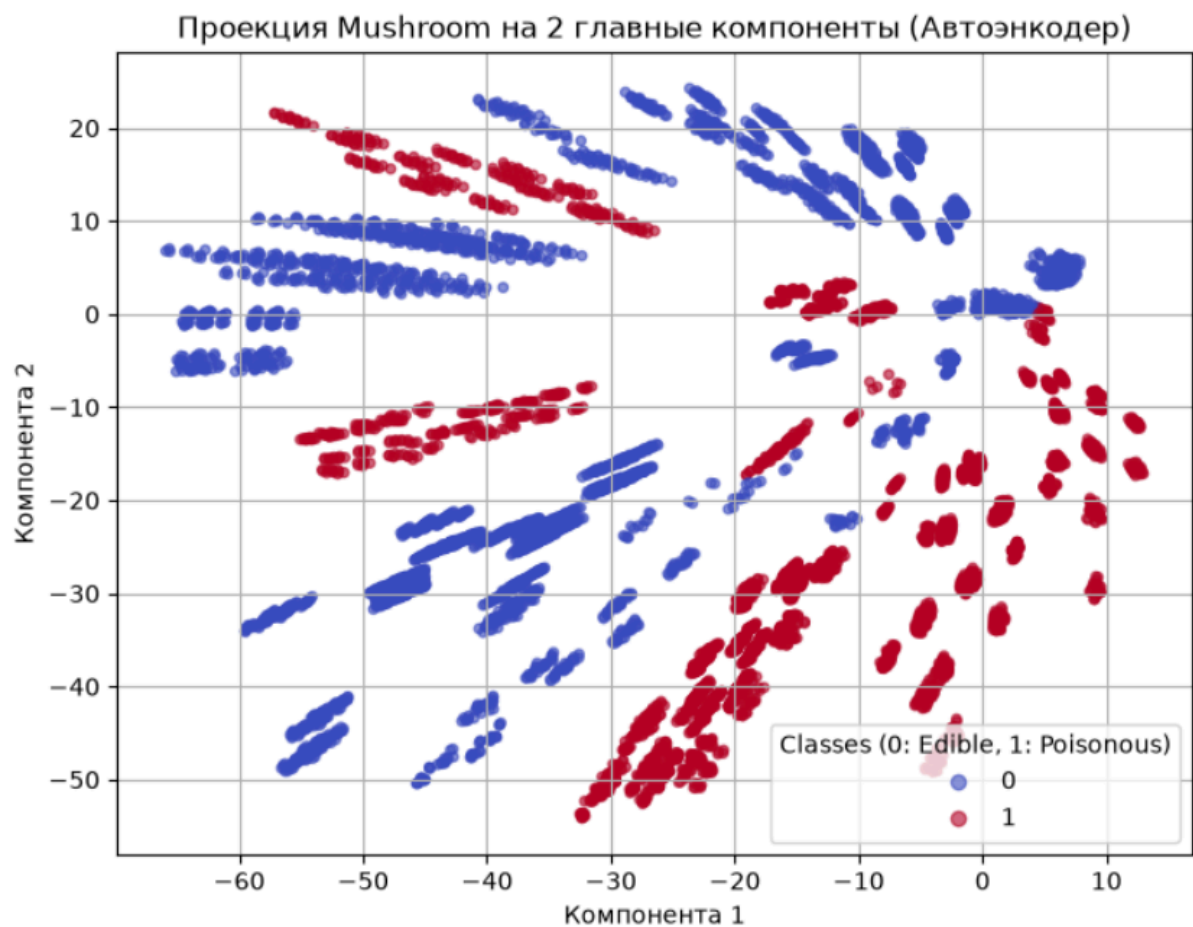
ax4.scatter(X_pca_3d[:, 0], X_pca_3d[:, 1], X_pca_3d[:, 2], c=y, cmap='coolwarm', alpha=0.6, s=15)

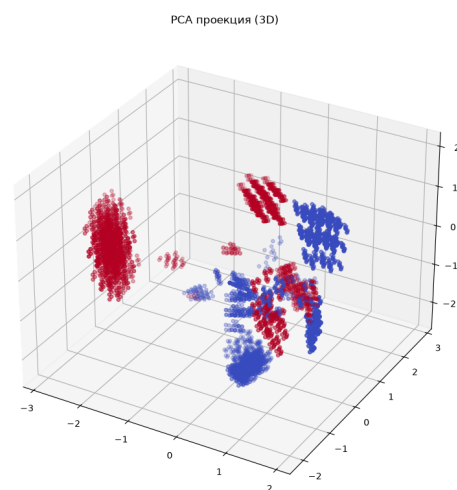
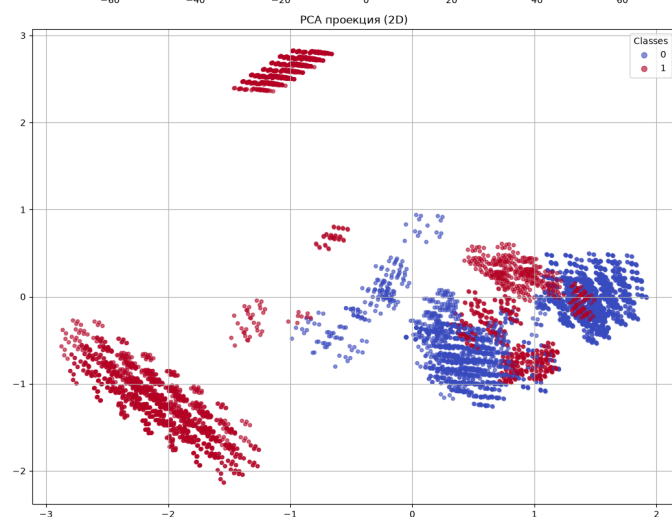
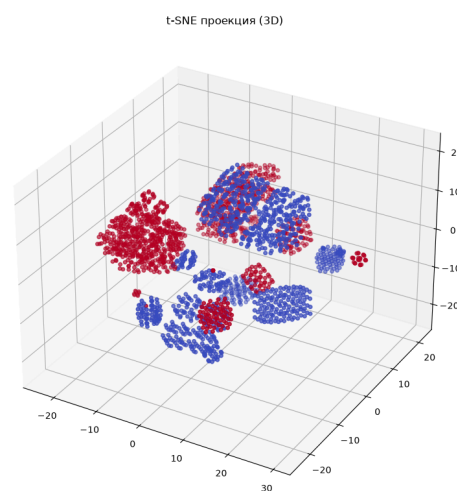
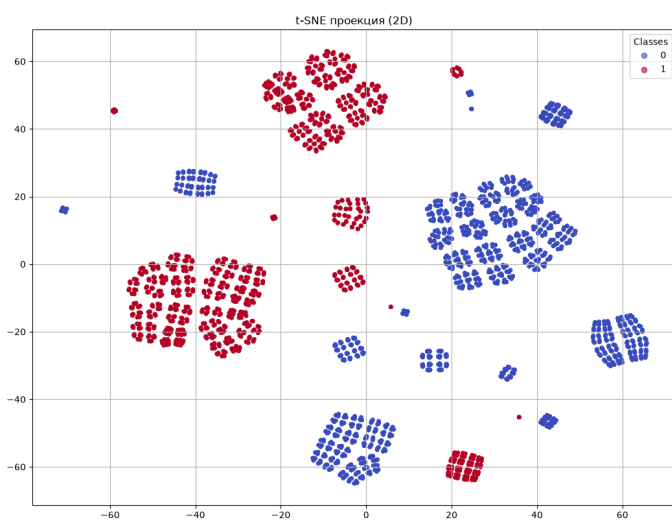
ax4.set_title('PCA проекция (3D)')

plt.tight_layout()

plt.show()
```

Результат работы





Вывод. Сравнение полученных проекций на наборе данных Mushrooms (где все исходные признаки являются категориальными и преобразованы через One-Hot Encoding) позволяет сделать следующие заключения об эффективности алгоритмов понижения размерности:

1. PCA (Метод главных компонент): Из-за своей линейной природы алгоритм пытается найти направления максимальной дисперсии во всем 117-мерном пространстве сразу. На бинарных (ONE) данных это приводит к тому, что точки выстраиваются в жесткие геометрические структуры (решетки и лучи), а классы (съедобные и ядовитые) сильно перемешиваются и накладываются друг на друга в центре. Проекция PCA в данном случае наименее информативна для визуального разделения.

2. t-SNE (Стохастическое вложение соседей с t-распределением): Нелинейный метод, оптимизирующий сохранение локальных расстояний (ближайших соседей). На категориальных данных t-SNE показывает выдающиеся результаты: он разбивает датасет на четкие, хорошо изолированные микро-кластеры. Классы "съедобный" и "ядовитый" практически идеально отделяются друг от друга, что позволяет визуально оценить сложность решаемой задачи классификации. Однако t-SNE искажает глобальные расстояния (расстояние между двумя далекими кластерами на графике t-SNE ничего не значит).
3. Сравнение с нейросетевым Автоэнкодером (задание 1): Автоэнкодер занимает промежуточное положение. За счет нелинейных функций активации в скрытых слоях он способен разделять классы лучше, чем PCA, но сохраняет при этом общую топологию и плотность данных (выстраивая их в характерные "полосы"), в отличие от t-SNE, который искусственно "разрывает" пространство на отдельные острова.